

Informe Final de Cumplimiento: Sistema de Préstamo de Equipos Tecnológicos (MVP)

Enlaces de Acceso y Evidencias del Proyecto

- **Repositorio del Proyecto:** [Repositorio del proyecto](#)
 - **Tablero de Gestión Ágil:** [Tablero del Proyecto](#)
 - **Evidencias en Video y Demostración:** [Enlace a Grabaciones y Demostración Funcional](#)
-

MIEMBROS DEL EQUIPO SCRUM

- **Product Owner:** - **Juan David Esparza Castillo** Representa las necesidades del cliente institucional, define y prioriza el Product Backlog, establece y aclara los criterios de aceptación de las historias de usuario y valida el incremento funcional durante la Sprint Review.
- **Scrum Master:** - **Duban Camilo Bernal** Facilita la adopción del marco Scrum, coordina los eventos del Sprint (Planning, Dailies, Review, Retrospective), gestiona la eliminación de impedimentos y fomenta la mejora continua del equipo.
- **Developers** - **Karen Yaritza Escobar Pinto** Analiza las historias de usuario, diseña la arquitectura modular, programa el software en Python, ejecuta pruebas funcionales, integra el código y asegura la persistencia de datos.

1. Resumen Ejecutivo y Propósito del MVP

El presente proyecto consistió en el desarrollo incremental de un Producto Mínimo Viable (MVP) en Python para dar solución a la gestión manual de préstamos de equipos tecnológicos en una institución educativa. Durante un Sprint de una semana, el equipo implementó una solución por consola que automatiza el registro de inventario, el padrón de estudiantes, el control de disponibilidad de equipos y la trazabilidad de préstamos y devoluciones con persistencia en JSON, aplicando integralmente el marco de trabajo Scrum.

2. Cumplimiento de Criterios Metodológicos y Marco Scrum

2.1. Aplicación de Roles, Eventos y Artefactos (Criterios 1, 4, 8 y 9)

- **Roles Scrum:** Se delimitaron y ejercieron las responsabilidades del Product Owner (priorización y validación de historias frente a criterios de aceptación), Scrum Master (facilitación de ceremonias y remoción de bloqueos) y Developers (análisis, arquitectura, programación y pruebas del incremento).
- **Eventos Ágiles Ejecutados:**
 - **Sprint Planning:** Definición formal del *Sprint Goal*, selección de historias del Product Backlog y estimación relativa de esfuerzo.
 - **Daily Scrum:** Cinco sesiones diarias de 5 minutos enfocadas en sincronización de avances y detección temprana de bloqueos, debidamente registradas en actas y videos.
 - **Sprint Review:** Demostración en vivo del MVP ejecutable ante el Product Owner, validando cada historia frente a su Criterio de Aceptación y obteniendo la aprobación formal del incremento funcional.
 - **Sprint Retrospective:** Análisis del desempeño del equipo respondiendo las preguntas clave del proceso y estableciendo tres acciones concretas de mejora para el próximo ciclo.
 - **Documentación:** Todas las actas oficiales de Planning, Dailies, Review y Retrospectiva se encuentran en formato PDF dentro de sus respectivas carpetas en el repositorio.

2.2. Gestión del Backlog y Planificación (Criterios 2 y 3)

- **Product Backlog:** Se estructuró un catálogo de historias de usuario priorizadas (HU01 a HU04) con criterios de aceptación explícitos y estimación relativa en Story Points.
- **Sprint Goal:** "Entregar un MVP funcional e integrado en Python que permita registrar inventario de equipos y estudiantes, gestionar préstamos y devoluciones en tiempo real con validación de disponibilidad y persistencia en JSON, y consultar reportes de equipos prestados e historial."
- **Sprint Backlog y Alcance:** El equipo comprometió y completó **13 Story Points** correspondientes a las historias transaccionales críticas (HU01 a HU04), aplazando las historias HU05 - HU08 para una versión posterior según la priorización del Product Owner.

2.3. Herramienta de Gestión y Trazabilidad (Criterios 4 y 5)

- Se mantuvo un tablero ágil digital visualizando el flujo continuo de trabajo a través de los estados: *Product Backlog*, *To Do*, *In Progress*, *Review/Test* y *Done*.
- Se gestionó una bitácora de impedimentos donde se resolvieron problemas de inicialización de datos en frío, desambiguación en devoluciones y validación de tipos de entrada en consola, evitando retrasos en la entrega del incremento.

3. Cumplimiento de Criterios Técnicos y Calidad del Software

3.1. Arquitectura Modular y Desarrollo en Python (Criterio 6)

La solución fue construida siguiendo principios de modularidad, alta cohesión y nombres descriptivos, distribuyendo las responsabilidades de la siguiente manera:

- `main.py` : Punto de entrada que gestiona el menú principal y la interacción con el usuario en consola.
- `equipos.py` : Módulo encargado del registro de dispositivos, validación de códigos únicos y consulta de inventario/disponibilidad.
- `estudiantes.py` : Módulo para la captura, validación de documentos y gestión de estudiantes institucionales.
- `prestamos.py` : Módulo que implementa las reglas de negocio: validación cruzada de existencia, bloqueo de préstamos sobre equipos ocupados, marcas temporales y transiciones atómicas de estado.
- `archivos.py` : Capa de persistencia desacoplada responsable de la lectura, escritura y manejo seguro de excepciones sobre los archivos `equipos.json`, `estudiantes.json` y `prestamos.json` en la carpeta `datos/`.

3.2. Calidad, Pruebas y Validación Funcional (Criterio 7)

- **Validaciones de Negocio:** El software previene la duplicidad de identificadores, rechaza entradas vacías o inválidas y garantiza que ningún equipo pueda ser prestado si se encuentra previamente ocupado.
- **Resultados de Pruebas:** Se ejecutó un plan de pruebas funcionales cubriendo el 100% de los flujos del sistema (registro de equipos/estudiantes, validación de no disponibilidad, préstamos exitosos, devoluciones con actualización de estado y consultas de historial), superando satisfactoriamente todos los casos evaluados sin fallas críticas.
- **Documentación de Pruebas:** El plan detallado y las evidencias de ejecución se encuentran organizados en el informe de pruebas del repositorio.

4. Organización del Repositorio y Entregables (Criterios 10 y 11)

El proyecto cumple estrictamente con la estructura de carpetas requerida por el instrumento de evaluación:

- [01_Product_Backlog/](#) : Documento de historias de usuario, priorización, criterios de aceptación y Story Points.
 - [02_Sprint_Planning/](#) : Acta del Sprint Planning, definición del Sprint Goal y desglose del Sprint Backlog.
 - [03_Daily_Scrum/](#) : Actas consolidadas de las reuniones diarias de sincronización.
 - [04_Impedimentos/](#) : Matriz de registro, seguimiento y resolución de bloqueos técnicos.
 - [05_Sprint_Review/](#) : Acta de revisión y validación de historias aceptadas por el Product Owner.
 - [06_Retrospectiva/](#) : Acta con el análisis del equipo, causas identificadas y las 3 acciones de mejora acordadas.
 - [07_Pruebas/](#) : Plan de casos de prueba.
 - [08_Informe_Final/](#) : Este reporte consolidado de cumplimiento ([Reporte_Final.md](#)).
 - [09_Codigo/](#) : Código fuente modular en Python ([main.py](#) , módulos funcionales y persistencia JSON).
 - [10_Videos/](#) : Enlaces directos a las carpetas y archivos en Google Drive con los videos de Daily Scrum y la demostración funcional del MVP.
 - [README.md](#) : Documentación raíz con descripción del sistema, prerequisites e instrucciones de ejecución.
-

5. Conclusiones y Balance de Evaluación

1. **Cumplimiento Integral de Objetivos:** El equipo entregó un MVP completamente funcional y alineado con la situación problemática planteada, validando el ciclo completo de préstamo y devolución institucional.
2. **Rigor Metodológico:** La aplicación de Scrum permitió mantener una visibilidad continua del progreso, gestionar impedimentos a tiempo y asegurar que el incremento cumpliera rigurosamente con la Definición de Hecho (DoD).
3. **Calidad Técnica:** La arquitectura modular y la persistencia desacoplada en JSON garantizan un código ordenado, mantenible y listo para extenderse en futuros sprints de desarrollo.